PYTHON PROGRAMMING

ACTIVITY-BASED LEARNING

10 Selected Case Studies with Complete Python Solutions

Name	T. Dhanush
Roll No.	26000136
Section	CSE-2
Subject	Python Programming

Prepared using the case-study options in the supplied Python Activity-Based Learning document.

Selected Topics and Syllabus Coverage

The following ten case studies were selected from the supplied activity sheet. They use only introductory Python concepts from the assigned syllabus: variables, data types, operators, conditionals, loops, strings, lists, nested lists, tuples and tuple unpacking.

1. University Scholarship Allocation

Variables, data types, operators, conditional statements, lists, loops, sorting

2. Progressive Electricity Billing

Variables, arithmetic operators, conditional statements, functions, loops

3. Employee Payroll and Deduction Analysis

Variables, arithmetic operators, conditional statements, formatted output

4. Retail Billing with Discount Rules

Lists, tuples, loops, operators, conditional statements

5. Examination Result and Classification

Lists, loops, conditional statements, arithmetic operators

6. Attendance Recovery Computation

Variables, operators, conditional statements, while loop

7. ATM Transaction Sequence Simulation

Lists, loops, conditional statements, operators

8. Student Marks Matrix Analysis

Nested lists, nested loops, lists, operators, conditional statements

9. Tuple-Based Product Catalogue Audit

Tuples, tuple unpacking, lists, loops, conditional statements

10. Student Feedback Sentiment Scoring

Strings, lists, loops, conditional statements

Submission note: The activity sheet also asks students to upload materials to a GitHub repository named Python-Activity-Based-Learning. Add actual output screenshots after running the programs in your Python environment.

1. University Scholarship Allocation

Syllabus topics: Variables, data types, operators, conditional statements, lists, loops, sorting

Problem Statement

Evaluate applicants using aggregate marks, subject-wise minimum scores, attendance and family-income criteria. Rank eligible applicants by aggregate marks, then attendance.

Objectives

Apply multiple eligibility conditions, store applicant records in a list, and rank qualifying students using clear tie-breaking rules.

Assumptions

Each applicant is represented as (name, aggregate, minimum_subject_mark, attendance, annual_income). Eligibility requires aggregate >= 75, every subject minimum >= 60, attendance >= 75%, and income <= ■250000.

Algorithm

1. Store applicant records. 2. Check each applicant against all four conditions. 3. Add eligible applicants to a list. 4. Sort by aggregate descending, then attendance descending, then name ascending. 5. Display the ranked list.

Flowchart (text form)

START → Read/store applicants → Check all eligibility conditions → Eligible? → Add to shortlist → Sort shortlist → Display ranking → END

Pseudocode

```
FOR each applicant:
    IF aggregate >= 75 AND subject_min >= 60
       AND attendance >= 75 AND income <= 250000:
        add applicant to eligible list
SORT by aggregate descending, attendance descending, name ascending
DISPLAY eligible list
```

Python Source Code

```
applicants = [
    ("Asha", 88, 65, 92, 180000),
    ("Bina", 79, 61, 80, 220000),
    ("Charu", 91, 58, 95, 150000),
    ("Diya", 79, 66, 85, 240000),
    ("Esha", 84, 70, 72, 120000)
]

eligible = []
for person in applicants:
    name, aggregate, subject_min, attendance, income = person
    if (aggregate >= 75 and subject_min >= 60
            and attendance >= 75 and income <= 250000):
        eligible.append(person)

eligible.sort(key=lambda x: (-x[1], -x[3], x[0]))
print("Scholarship shortlist:")
for rank, person in enumerate(eligible, start=1):
    print(rank, person[0], "Aggregate:", person[1],
          "Attendance:", person[3])
```

Test Case / Expected Result

Expected eligible ranking: 1. Asha (88, 92%); 2. Diya (79, 85%); 3. Bina (79, 80%). Charu fails subject minimum; Esha fails attendance.

Conclusion

The program applies all eligibility rules and produces a deterministic ranked shortlist.

2. Progressive Electricity Billing

Syllabus topics: Variables, arithmetic operators, conditional statements, functions, loops

Problem Statement

Calculate a household electricity bill using progressive consumption slabs, a fixed charge, and a rebate for low consumption.

Objectives

Use if/elif logic to calculate slab charges and combine variable and fixed costs.

Assumptions

First 100 units cost ■2/unit; next 100 cost ■3/unit; units above 200 cost ■5/unit. Fixed charge is ■50. Consumption of 100 units or less receives a ■20 rebate. Units cannot be negative.

Algorithm

1. Read units. 2. Reject negative input. 3. Calculate charges slab by slab. 4. Add fixed charge. 5. Apply rebate if eligible. 6. Print bill.

Flowchart (text form)

START → Input units → Units valid? → Calculate slab charges → Add fixed fee → Apply rebate → Print total → END

Pseudocode

```
IF units < 0: report invalid input
ELSE:
    charge first 100 units at ■2
    charge next 100 units at ■3
    charge remaining units at ■5
    total = energy charge + 50
    IF units <= 100: total = total - 20
    DISPLAY total
```

Python Source Code

```
def energy_charge(units):
    if units <= 100:
        return units * 2
    elif units <= 200:
        return 100 * 2 + (units - 100) * 3
    else:
        return 100 * 2 + 100 * 3 + (units - 200) * 5

units = int(input("Enter units consumed: "))
if units < 0:
    print("Invalid consumption")
else:
    total = energy_charge(units) + 50
    if units <= 100:
        total -= 20
    print("Energy charge: Rs.", energy_charge(units))
    print("Fixed charge: Rs. 50")
    print("Rebate: Rs.", 20 if units <= 100 else 0)
    print("Total bill: Rs.", total)
```

Test Case / Expected Result

Input: 250 units. Energy charge = 100×2 + 100×3 + 50×5 = ■750; fixed charge = ■50; rebate = ■0; total = ■800.

Conclusion

The program calculates progressive charges correctly and validates invalid negative consumption.

3. Employee Payroll and Deduction Analysis

Syllabus topics: Variables, arithmetic operators, conditional statements, formatted output

Problem Statement

Calculate gross salary from basic pay, allowances, overtime and incentives, then apply a simple deduction policy.

Objectives

Calculate salary components and produce a readable salary statement.

Assumptions

HRA is 20% of basic pay, DA is 10%, overtime is paid at ■150/hour, and incentives are entered directly. Deduction is 5% of gross salary when gross salary exceeds ■30000; otherwise it is 2%.

Algorithm

1. Read basic pay, overtime hours and incentive. 2. Validate non-negative values. 3. Calculate HRA, DA and overtime pay. 4. Add to obtain gross salary. 5. Calculate deduction rate. 6. Print net salary.

Flowchart (text form)

START → Input salary details → Validate → Calculate allowances/overtime → Gross salary → Apply deduction rate → Net salary statement → END

Pseudocode

```
HRA = basic × 0.20
DA = basic × 0.10
overtime_pay = hours × 150
gross = basic + HRA + DA + overtime_pay + incentive
IF gross > 30000: deduction = gross × 0.05
ELSE: deduction = gross × 0.02
net = gross - deduction
```

Python Source Code

```
basic = float(input("Basic pay (Rs.): "))
hours = float(input("Overtime hours: "))
incentive = float(input("Incentive (Rs.): "))

if basic < 0 or hours < 0 or incentive < 0:
    print("Invalid input: values cannot be negative")
else:
    hra = basic * 0.20
    da = basic * 0.10
    overtime_pay = hours * 150
    gross = basic + hra + da + overtime_pay + incentive
    rate = 0.05 if gross > 30000 else 0.02
    deduction = gross * rate
    net = gross - deduction

    print("\nSALARY STATEMENT")
    print("Basic pay: Rs.", round(basic, 2))
    print("HRA: Rs.", round(hra, 2))
    print("DA: Rs.", round(da, 2))
    print("Overtime: Rs.", round(overtime_pay, 2))
    print("Incentive: Rs.", round(incentive, 2))
    print("Gross salary: Rs.", round(gross, 2))
    print("Deduction: Rs.", round(deduction, 2))
    print("Net salary: Rs.", round(net, 2))
```

Test Case / Expected Result

Input: basic ■25000, overtime 4 hours, incentive ■1000. HRA ■5000, DA ■2500, overtime ■600, gross ■34100, deduction ■1705, net ■32395.

Conclusion

The salary statement separates earnings, deductions and take-home pay.

4. Retail Billing with Discount Rules

Syllabus topics: Lists, tuples, loops, operators, conditional statements

Problem Statement

Generate an itemized invoice for products, applying a threshold discount and a fixed tax rate.

Objectives

Process multiple items using a list of tuples, calculate line totals, and apply bill-level rules.

Assumptions

Each product is (name, price, quantity). A 10% discount applies when subtotal is at least ■2000. GST is assumed to be 5% of the discounted subtotal. This is a classroom example, not a tax invoice.

Algorithm

1. Store product tuples. 2. Calculate each line total and subtotal. 3. Apply discount threshold. 4. Calculate tax on discounted subtotal. 5. Print itemized invoice and final amount.

Flowchart (text form)

START → Read product tuples → Calculate line totals → Subtotal → Check discount threshold → Add tax → Print invoice → END

Pseudocode

```
subtotal = 0
FOR each (name, price, quantity):
    line_total = price × quantity
    subtotal = subtotal + line_total
IF subtotal >= 2000: discount = 10% of subtotal
ELSE: discount = 0
tax = 5% of (subtotal - discount)
final = subtotal - discount + tax
```

Python Source Code

```
products = [
    ("Notebook", 80, 5),
    ("Bag", 900, 1),
    ("Pen pack", 120, 3)
]

subtotal = 0
print("ITEM        PRICE  QTY  TOTAL")
for name, price, quantity in products:
    line_total = price * quantity
    subtotal += line_total
    print(name, price, quantity, line_total)

discount = subtotal * 0.10 if subtotal >= 2000 else 0
taxable_amount = subtotal - discount
tax = taxable_amount * 0.05
final_amount = taxable_amount + tax

print("Subtotal: Rs.", round(subtotal, 2))
print("Discount: Rs.", round(discount, 2))
print("Tax (5%): Rs.", round(tax, 2))
print("Amount payable: Rs.", round(final_amount, 2))
```

Test Case / Expected Result

Subtotal = 400 + 900 + 360 = ■1660. No discount; tax = ■83; final amount = ■1743.

Conclusion

Lists and tuples make it convenient to store and process itemized billing records.

5. Examination Result and Classification

Syllabus topics: Lists, loops, conditional statements, arithmetic operators

Problem Statement

Calculate total, average and pass/fail status from subject marks, using a subject-wise pass mark.

Objectives

Process a list of marks, check each subject, and assign a result classification.

Assumptions

There are five subjects. Each mark must be between 0 and 100. A student passes only if every subject mark is at least 35. Classification for passed students: Distinction at average >= 75, First Class at >= 60, otherwise Pass.

Algorithm

1. Store marks. 2. Validate each mark. 3. Calculate total and average. 4. Check every subject for pass mark. 5. Assign classification and display results.

Flowchart (text form)

START → Read marks → Validate range → Calculate total/average → All subjects >= 35? → Classify → Display result → END

Pseudocode

```
IF any mark is outside 0..100: report invalid marks
ELSE:
    total = sum(marks)
    average = total / number of subjects
    IF any mark < 35: result = FAIL
    ELSE IF average >= 75: result = DISTINCTION
    ELSE IF average >= 60: result = FIRST CLASS
    ELSE: result = PASS
```

Python Source Code

```
marks = [82, 76, 69, 88, 73]

valid = True
for mark in marks:
    if mark < 0 or mark > 100:
        valid = False

if not valid:
    print("Invalid marks: each mark must be 0 to 100")
else:
    total = sum(marks)
    average = total / len(marks)
    if any(mark < 35 for mark in marks):
        result = "FAIL"
    elif average >= 75:
        result = "DISTINCTION"
    elif average >= 60:
        result = "FIRST CLASS"
    else:
        result = "PASS"

    print("Marks:", marks)
    print("Total:", total, "/", len(marks) * 100)
    print("Average:", round(average, 2))
    print("Result:", result)
```

Test Case / Expected Result

Marks [82, 76, 69, 88, 73] give total 388/500, average 77.6 and result DISTINCTION. If any subject is below 35, result is FAIL.

Conclusion

The program distinguishes subject-wise eligibility from overall average classification.

6. Attendance Recovery Computation

Syllabus topics: Variables, operators, conditional statements, while loop

Problem Statement

Calculate current attendance percentage and find the minimum consecutive classes a student must attend to reach a target percentage.

Objectives

Use a loop to solve a changing-percentage problem rather than guessing the number of classes.

Assumptions

The student has attended 36 of 50 classes. The target is 75%. Assume every future class is attended.

Algorithm

1. Read attended and conducted classes. 2. Validate values. 3. Calculate current percentage. 4. While the projected percentage is below target, add one attended class and one conducted class. 5. Report the number needed.

Flowchart (text form)

START → Input attended/conducted → Validate → Compute current % → Target reached? → Add one attended class and one total class → Repeat → Display classes needed → END

Pseudocode

```
needed = 0
WHILE attended / conducted × 100 < target:
    attended = attended + 1
    conducted = conducted + 1
    needed = needed + 1
DISPLAY needed
```

Python Source Code

```
attended = 36
conducted = 50
target = 75
original_attended = attended
original_conducted = conducted

if conducted <= 0 or attended < 0 or attended > conducted:
    print("Invalid attendance values")
else:
    current = attended / conducted * 100
    needed = 0
    while attended / conducted * 100 < target:
        attended += 1
        conducted += 1
        needed += 1

    print("Current attendance:", round(current, 2), "%")
    print("Consecutive classes to attend:", needed)
    print("Projected attendance:",
          round(attended / conducted * 100, 2), "%")
```

Test Case / Expected Result

Current attendance = 36/50 × 100 = 72%. After 6 attended classes, attendance becomes 42/56 × 100 = 75%. Required classes = 6.

Conclusion

A while loop finds the smallest number of future classes needed to reach the target.

7. ATM Transaction Sequence Simulation

Syllabus topics: Lists, loops, conditional statements, operators

Problem Statement

Process deposits and withdrawals, reject invalid transactions, prevent overdrafts, and summarize the final balance.

Objectives

Simulate a sequence of transactions while maintaining a valid account balance.

Assumptions

Opening balance is ■2000. Transactions are tuples of type and amount. Deposits and withdrawals must be positive; withdrawals cannot exceed the available balance.

Algorithm

1. Set opening balance. 2. Process each transaction tuple. 3. Validate amount. 4. Apply valid deposits or withdrawals. 5. Record accepted and rejected transactions. 6. Display final balance and counts.

Flowchart (text form)

START → Set opening balance → Read each transaction → Valid amount/type and sufficient funds? → Update balance or reject → Next transaction → Summary → END

Pseudocode

```
FOR each (kind, amount):
    IF amount <= 0: reject
    ELSE IF kind is deposit: balance = balance + amount
    ELSE IF kind is withdrawal AND amount <= balance:
        balance = balance - amount
    ELSE: reject
DISPLAY balance and transaction counts
```

Python Source Code

```
balance = 2000
transactions = [
    ("deposit", 500),
    ("withdraw", 700),
    ("withdraw", 2500),
    ("deposit", -100),
    ("withdraw", 300)
]
accepted = 0
rejected = 0

for kind, amount in transactions:
    if amount <= 0:
        print("Rejected:", kind, amount, "- amount must be positive")
        rejected += 1
    elif kind == "deposit":
        balance += amount
        accepted += 1
        print("Deposit accepted:", amount)
    elif kind == "withdraw" and amount <= balance:
        balance -= amount
        accepted += 1
        print("Withdrawal accepted:", amount)
    else:
        print("Rejected:", kind, amount, "- insufficient funds")
        rejected += 1

print("Final balance: Rs.", balance)
print("Accepted transactions:", accepted)
print("Rejected transactions:", rejected)
```

Test Case / Expected Result

Opening ■2000; +■500 → ■2500; –■700 → ■1800; ■2500 withdrawal rejected; negative deposit rejected; –■300 → final balance ■1500. Accepted = 3, rejected = 2.

Conclusion

The simulation enforces balance and transaction rules before changing account state.

8. Student Marks Matrix Analysis

Syllabus topics: Nested lists, nested loops, lists, operators, conditional statements

Problem Statement

Analyse marks for several students across multiple subjects, calculate student totals and subject averages, and identify the top performer.

Objectives

Use a nested list to represent a marks matrix and compute row-wise and column-wise statistics.

Assumptions

Rows represent students and columns represent three subjects. Each mark is between 0 and 100.

Algorithm

1. Store marks in a nested list. 2. Calculate each student's total. 3. Find the highest total. 4. For each subject, add marks across students and divide by student count. 5. Display results.

Flowchart (text form)

START → Read marks matrix → Loop through rows for totals → Find top student → Loop through columns for averages → Display summary → END

Pseudocode

```
FOR each student row:
    total = sum(row)
    store total
best_index = index of maximum total
FOR each subject column:
    subject_total = sum(matrix[row][column])
    average = subject_total / number of students
DISPLAY totals, top student and averages
```

Python Source Code

```
students = ["Asha", "Bina", "Charu"]
subjects = ["Python", "Maths", "English"]
marks = [
    [85, 78, 90],
    [92, 88, 84],
    [76, 91, 80]
]

totals = []
for row in marks:
    totals.append(sum(row))

best_index = totals.index(max(totals))
print("Student totals:")
for i in range(len(students)):
    print(students[i], ":", totals[i])

print("Top performer:", students[best_index])
print("Subject averages:")
for col in range(len(subjects)):
    total = 0
    for row in range(len(marks)):
        total += marks[row][col]
    average = total / len(marks)
    print(subjects[col], ":", round(average, 2))
```

Test Case / Expected Result

Totals: Asha 253, Bina 264, Charu 247. Top performer: Bina. Averages: Python 84.33, Maths 85.67, English 84.67.

Conclusion

Nested lists represent tabular data and nested loops support row-wise and column-wise analysis.

9. Tuple-Based Product Catalogue Audit

Syllabus topics: Tuples, tuple unpacking, lists, loops, conditional statements

Problem Statement

Audit a product catalogue for invalid prices or stock, calculate inventory value, and identify items needing replenishment.

Objectives

Use immutable tuples to represent product records and unpack tuple fields during processing.

Assumptions

Each record is (product_id, price, stock). Price and stock must be non-negative. Replenishment is needed when stock is below 5 units.

Algorithm

1. Store product tuples. 2. Unpack each tuple. 3. Flag invalid records. 4. Calculate price × stock for valid records. 5. Add low-stock items to a replenishment list. 6. Display results.

Flowchart (text form)

START → Read tuple catalogue → Unpack each record → Validate fields → Calculate stock value → Check stock < 5 → Print audit → END

Pseudocode

```
FOR each (product_id, price, stock):
    IF price < 0 OR stock < 0: flag invalid
    ELSE:
        inventory_value += price × stock
        IF stock < 5: add product_id to replenishment list
DISPLAY inventory value and flagged products
```

Python Source Code

```
catalogue = [
    ("P101", 120.0, 12),
    ("P102", 75.0, 3),
    ("P103", 250.0, 0),
    ("P104", -10.0, 8)
]

inventory_value = 0
replenish = []
invalid = []

for product_id, price, stock in catalogue:
    if price < 0 or stock < 0:
        invalid.append(product_id)
    else:
        inventory_value += price * stock
        if stock < 5:
            replenish.append(product_id)

print("Inventory value: Rs.", round(inventory_value, 2))
print("Items to replenish:", replenish)
print("Invalid records:", invalid)
```

Test Case / Expected Result

Valid inventory value = P101 ■1440 + P102 ■225 + P103 ■0 = ■1665. Replenish P102 and P103. Invalid record: P104.

Conclusion

Tuple unpacking makes structured product records easy to read while validation prevents invalid values from affecting totals.

10. Student Feedback Sentiment Scoring

Syllabus topics: Strings, lists, loops, conditional statements

Problem Statement

Score feedback sentences by counting words from supplied positive and negative keyword lists, then classify each sentence.

Objectives

Use basic string operations and list membership to classify sample feedback.

Assumptions

Matching is case-insensitive and punctuation is removed using simple replacements. Each occurrence of a listed keyword contributes one point. Score = positive count – negative count. Scores above zero are Positive, below zero Negative, and zero Neutral.

Algorithm

1. Define positive and negative word lists. 2. Convert each sentence to lowercase. 3. Replace common punctuation with spaces. 4. Split into words. 5. Count matching words. 6. Calculate score and classify.

Flowchart (text form)

START → Read feedback → Normalize case/punctuation → Split into words → Count positive/negative terms → Calculate score → Classify → END

Pseudocode

```
FOR each sentence:
    normalize sentence
    words = split sentence into words
    positive = count words in positive list
    negative = count words in negative list
    score = positive - negative
    IF score > 0: Positive
    ELSE IF score < 0: Negative
    ELSE: Neutral
```

Python Source Code

```
positive_words = ["good", "helpful", "clear", "excellent"]
negative_words = ["bad", "confusing", "slow", "poor"]
feedback = [
    "The lessons are clear and helpful!",
    "The website is slow and confusing.",
    "The course is good."
]
punctuation = ".,!?;:"

for sentence in feedback:
    cleaned = sentence.lower()
    for mark in punctuation:
        cleaned = cleaned.replace(mark, " ")
    words = cleaned.split()

    positive = 0
    negative = 0
    for word in words:
        if word in positive_words:
            positive += 1
        if word in negative_words:
            negative += 1

    score = positive - negative
    if score > 0:
        label = "Positive"
    elif score < 0:
        label = "Negative"
    else:
        label = "Neutral"

    print(sentence)
    print("Positive:", positive, "Negative:", negative,
          "Score:", score, "Class:", label)
    print()
```

Test Case / Expected Result

1) 'clear and helpful' → score +2, Positive. 2) 'slow and confusing' → score −2, Negative. 3) 'good' → score +1, Positive.

Conclusion

The program demonstrates case normalization, splitting, keyword counting and rule-based text classification.